

MICROPROCESSOR → CISC - Intel pentium
RISC - ARM
Special purpose - GPU & JPU

CISC - shorter code (hardware complexity)
VLIW - very long int word (multiple small ins. together)
Variable length inst → compactness increases → ↑ code density
larger register set
4 programmed CU
may take many clock cycles for inst.

RISC (compiler complexity)
- code density - poor
32-bit fixed
load-store arch - Bon Nuemann arch
32 32-bit registers
Hardwired CU
pipelined
single cycle
simplify inst
smaller die size (less transistor - less power)
shorter development
faster - high performance

Diff b/w RISC & CISC → 2M
4 points.

Benchmark (speed bench)

processor best use by → pipelining
→ cache mem

→ super scalar - multiple inst. in 1 clock cycle

CISC

compiler



processor

complexity

RISC

compiler

complexity



processor

AHB

APB

arbiter

ARM (Acron RISC Machine)

(3-stage pipeline) ARM 7TMDI

→ FETCH : PC, MAR, MDR

→ DECODE : identify opcode & operands

→ EXECUTE : ALU

→ data path
→ control path

μ PROCESSOR → brain of comp
basic computing unit on an IC

- Instruction set — instr can execute
- Bandwidth — no. of bits processed in single inst.
- Clock speed — instructions per second executed (MHz)

CISC → Complex Instruction set computer

RISC → Reduced Instruction set computer

RISC

- Simple instruction
- Fixed length instr.
- Multiple register set
- 3 operands per instr.
- Parameter passing through register windows
- Single-cycle instr.
- Hardwired control
- Highly pipelined
- Only load/store can access memory
- Few addressing modes
- Complexity in compiler
- higher performance
- poor code density
- Do not execute x86 code

CISC

- Complex instruction
- Variable length instr.
- Single register set
- One or 2 register operands per instruction
- Parameter passing through memory
- Multiple cycle instr.
- Microprogrammed control
- Less pipelined
- Many inst. can access memory
- Many addressing modes

Slide - 2

ARM PROGRAM STRUCTURE

Address

Instruction & Data

Address of Instr. 1
Address of Instr. 2
⋮
Address of Instr. n

• text

ARM Instr. 1
ARM Instr. 2
⋮
ARM Instr. n

Address of Data 1
Address of Data n

• data

Declaration of variable 1
⋮
Declaration of var. n

GENERAL FORMAT:

MNEMONIC {condition} {S} {Rd}, Op1, Op2

↓
ADD, SUB

↓
EQ, EQ, MI

↓
S → results in condition
flag updation (N, Z, V, C)

EQ → Z = 1
NEQ → Z = 0

MI → N = 1
PL → N = 0

VS(overflow) → V = 1
VC → V = 0

N — negative

Z — zero

C — carry

V — overflow

I — IRQ → when disable = 1

F — FIQ → when disable = 1

T — 0 ARM state

1 Thumb state

R10 sl stack limit

R11 fp frame pointer → points to the base of current stack frame

R12 ip intra-procedure-call scratch register → general purpose — temporarily within a subroutine

R13 sp stack pointer

R14 lr link register

R15 pc program counter

Base	Prefix	Suffix
Binary	—	Y, y
Decimal	—	T, none
Hexa	0x/0X	H or h
Octal	0	Q, q, O, o

Q. Identify the values of the NCZV flags for the following snippet of code

(a) MOV r0, #-10
 MOV r1, #-10
 ADDS r2, r1, r0
 N C Z V
 1 1 0 0

$$\begin{array}{r}
 10 = 00001010 \\
 -10 = 11110110 \\
 \begin{array}{r}
 11110110 \\
 11110110 \\
 \hline
 11101100
 \end{array}
 \end{array}$$

(b) MOV r0, #-10
 MOV r1, #-10
 ADD r2, r1, r0

N C Z V
 0 0 0 0

since no 'S'
 condition

(c) MOV r0, #-44
 MOV r1, #4
 SUBS r2, r0, r1
 ADDMIS r0, r0, r1

$$r2 = -48$$

MIS $\rightarrow$ -ve $\rightarrow$ executed

N C Z V
 1 1 0 0
 1 0 0 0

(d) MOV r0, #44
 MOV r1, #4
 CMP r1, r0
 ADDEQ r0, r0, r1

N C Z V

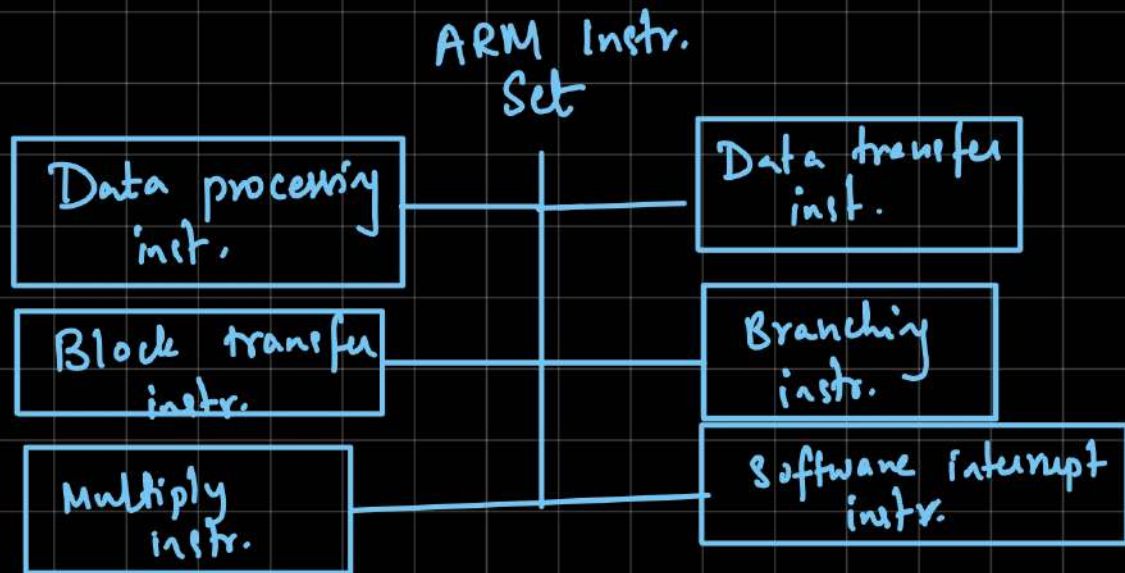
1 0 0 0 CMP

Not executed since N=1

0 1 1 0 CMP

if r0=44
 r1=44

- * For addition, CMN inst. $C = 1$ if add produce carry (unsigned)
 $C = 0$ otherwise
- * For subtraction, CMP inst. $C = 0$ if sub produce borrow
 $C = 1$ otherwise



DATA PROCESSING $\rightarrow$ MOV, ADD, SUB, CMP, CMN

BRANCH INST. $\rightarrow$ B, BL, BG, BGT, BLT, BGE, BLE

DATA TRANSFER $\rightarrow$ LOAD-STORE, LDR, STR, LDM, STM

SOFTWARE INTERRUPT $\rightarrow$ SWI, 0x11

PROGRAM STATUS REG $\rightarrow$ MSR, MRS

① DATA PROCESSING

MOVE { $\text{MOV } R0, R1 \rightarrow R0 \leftarrow R1$
 $\text{MVN } R0, R1 \rightarrow R0 \leftarrow \text{neg}(R1)$

Arith-metic { $\text{ADD } R3, R1, R2 \rightarrow R3 \leftarrow R1 + R2$
 $\text{SUB } R3, R1, R2 \rightarrow R3 \leftarrow R1 - R2$

compare { $\text{CMP } R3, R1 \rightarrow R3 - R1$
 $\text{CMN } R2, R0 \rightarrow R2 + R0$ } does not reflect in output but just the NCZV flags

COMPARISON

- compare : $CMP\ R1, R2 \rightarrow R1 - R2$
- compare negate : $CMN\ R1, R2 \rightarrow R1 + R2$
- bit test : $TST\ R1, R2 \rightarrow R1\ \text{and}\ R2$
- test equal : $TEQ\ R1, R2 \rightarrow R1\ \text{xor}\ R2$

ARITHMETIC OPERATIONS

$ADD\ r0, r1, r2 \rightarrow r0 = r1 + r2$

$ADC \quad r0 = r1 + r2 + C$

$SUB \quad r0 = r1 - r2$

$SBC \quad r0 = r1 - r2 + C - 1$

$RSB \quad r0 = r2 - r1\ (\text{reverse sub})$

$RSC \quad r0 = r2 - r1 + C - 1$

adding a S after inst $\rightarrow$ condition flags are set

BITWISE LOGICAL OPERATIONS

$AND\ r0, r1, r2 \rightarrow r0 = r1\ \text{and}\ r2$

$ORR\ r0, r1, r2 \quad r0 = r1\ \text{or}\ r2$

$EOR\ r0, r1, r2 \quad r0 = r1\ \text{xor}\ r2$

$BIC\ r0, r1, r2 \quad r0 = r1\ \text{and}\ (\text{not}\ r2)$
(bit clear)

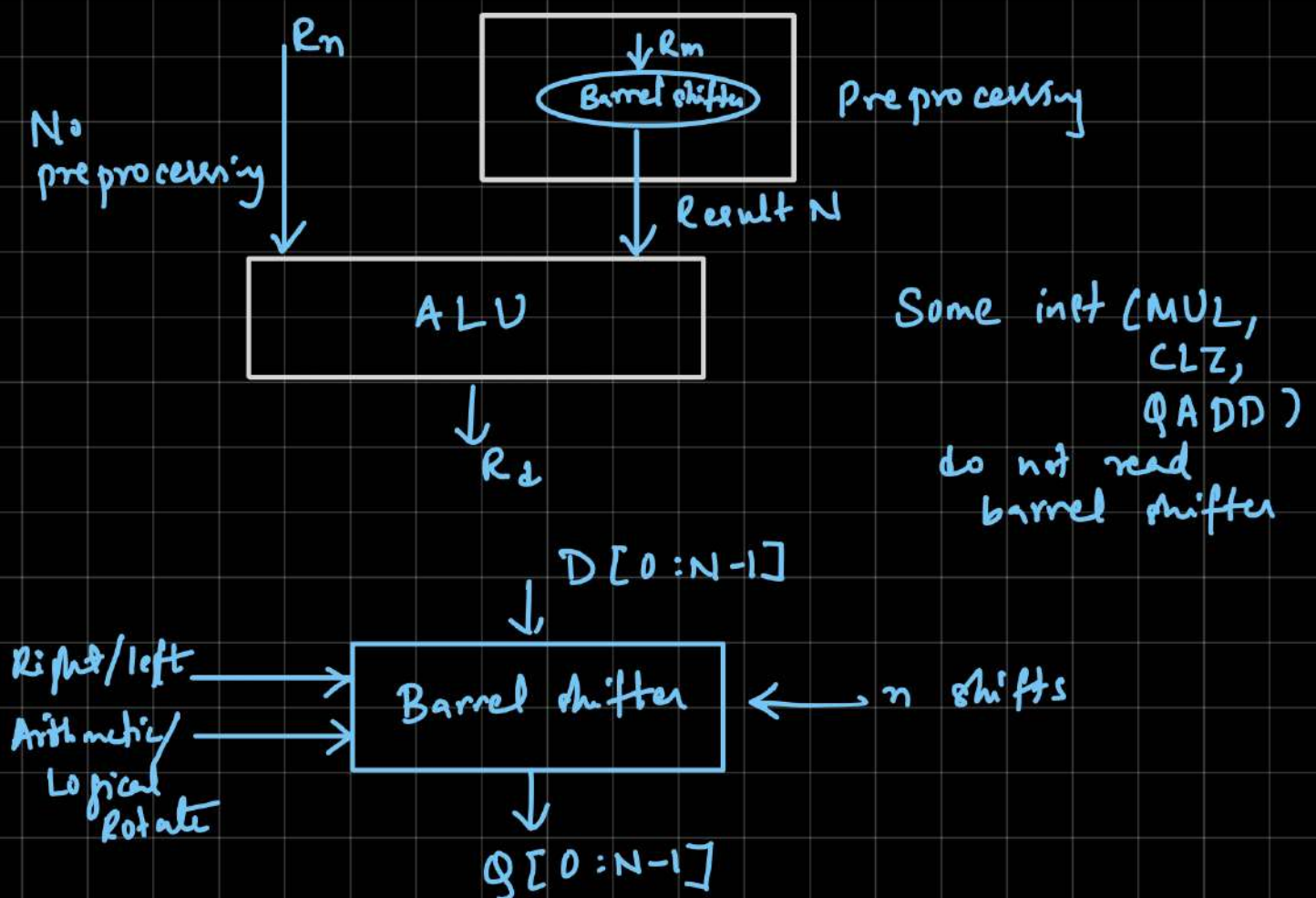
Q. Find o/p of following statements if R1, R2 are source reg & R0 is dest register.

$BIC\ R0, R1, R2$

R1	0100 0110	1010 0001	1111 0001	1011 0111
R2	1111 1111	1111 1111	0000 0000	0000 0000
$\overline{R2}$	0000 0000	0000 0000	1111 1111	1111 1111
R0	0000 0000	0000 0000	1111 0001	1011 0111

	R1	0100 0110	1010 0001	1111 0001	1011 0111
	R2	1111 1111	1111 1111	0000 0000	0000 0000
AND R3, R1, R2	R3	0100 0110	1010 0001	0000 0000	0000 0000
OR R4, R1, R2	R4	1111 1111	1111 1111	1111 0001	1011 0111
EOR R5, R1, R2	R5	1011 1001	0101 1110	1111 0001	1011 0111
BIC R6, R1, R2	R6	0000 0000	0000 0000	1111 0001	1011 0111
MVN R7, R2	R7	0000 0000	0000 0000	1111 1111	1111 1111

SHIFT REGISTER OPERANDS



* crossbar switch matrix architecture $\rightarrow$ to minimize delay
 $\nearrow$ contributes directly to datapath cycle time

* Arithmetic Left shift / Logical Left shift

1000 $\xrightarrow{LS}$ 0000

Add a zero at empty space

* Logical Right shift

1100 $\xrightarrow{RS}$ 0110

"

* Arithmetic Right shift

1101 $\rightarrow$ 1110

0001 $\rightarrow$ 0000

Sign extension

LSL — logical shift left

LSR — logical shift right

ASR — arth. right shift

ROR — rotate right

RRX — rotate right extended

x LSL y

x LSR y

x ASR y

x ROR y

x RRX

$x \ll y$

$x \gg y$

$x \gg y$

$x \gg y \mid x \ll (32 - y)$

$(\ll y \ll 31) \mid$

$x \gg 1$

Q. Find the o/p of the following inst. after performing logical L & R & Arith L & R

	LL	LR	AL	AR
① 101011	010110			
② 0110011				
③ 0xA01				

Q. MOV R0, #3 R0 - 0011
MOV R1, #4 R1 - 0100
ADDS R3, R0, R1, LSL #5

$$\begin{array}{r} 1000 \ 0000 \\ 0000 \ 0011 \\ \hline 1000 \ 0011 \rightarrow R3 \\ \#0x83 = \#131 \end{array}$$

* MULTIPLICATION BY 2^n (1, 2, 4, 8 ...)

MOV R_a, R_b, LSL #n

* MULTIPLICATION BY 2^{n+1} (3, 5, 9, 17...)

ADD Ra, Ra, Ra, LSL #n

* MULTIPLICATION BY 2^{n-1} (3, 7, 15, ...)

RSB $R_a, R_a, R_a, LSL \# n$

* MULTIPLICATION BY 6

ADD R_a, R_a, R_a, LSL #1 — multiply by 3

MOV R_a, R_a, LSL #1 - and then by 2

* MULTIPLICATION BY 10 & add in extra number

ADD R_a, R_a, R_a, LSL #2 - multiply by 5

ADD R_a, R_c, R_a, LSL #1 - multiply by 2
and add in next
digit (R_c)

Q. For multiplying a no by 35 and don't use mul.
inst.

ADD Ra,Ra,Ra,LSL #2

RSB Ra,Ra,Ra,LSL #3

N Shift Operations

Immediate
Register

LSL by imm.

LSL by reg

Rotate right by imm

Rotate right by reg

Rotate right by extend

Syntax

#immediate

R_m

$R_m, \text{LSL} \# \text{shift-imm}$

$R_m, \text{LSL} R_s$

$R_m, \text{LSR} \# \text{shift-imm}$

$R_m, \text{LSR} R_s$

$R_m, \text{ASR} \# \text{shift-imm}$

$R_m, \text{ASR} R_s$

$R_m, \text{ROR} \# \text{shift-imm}$

$R_m, \text{ROR} R_s$

R_m, RRX

② BRANCH / FLOW CONTROL INST.

$B \{ \text{cond} \} \text{Label}$

$BL \{ \text{cond} \} \text{Label}$

$BX \{ \text{cond} \} R_m$

$BLX \{ \text{cond} \} R_m$

* B - Branch - $PC = \text{Label}$

* BL - Branch & link - ① $R14 \leftarrow PC$

② $PC = \text{Label}$.

* BX - Branch exchange

* BLX - Branch exchange & link

ARM⁽³²⁾ to Thumb mode⁽¹⁶⁾ or
vice versa

Branch Inst

→ Unconditional = $B \text{ Label}$

→ Conditional = $BNE \text{ Label}$

Q.

```
.text
mov r1, #3
bl foo
add r2, r0, r1
add r2, r0, r1
add r0, r0, #1
swi 0x011
foo: mov r0, #2
bx lr
.end
```

Q. Write a program in A.L.L to compare 2 nos, perform add if no are equal, perform sub if not equal

Q. Program to find the sum of N numbers and store the result in memory location

Conditional Branch
Options

Q. Program to find the sum of N number using
half word

Byte

③ ADDRESSING MEMORY LOCATIONS

LDR R0, [R1] @ mem[R1]

① Immediate

LDR R0, [R1, #4] @ mem[R1+4]

② Register

LDR R0, [R1, R2] @ mem[R1+R2]

③ Scaled Register

LDR R0, [R1, R2, LSL #2] @ mem[R1+4R2]

ADDRESSING MODES
USED IN ARM

PRE INDEXING WITHOUT WRITE BACK

LDR Rd, [Rn, offset]

PRE INDEXING WITH WRITE BACK

LDR Rd, [Rn, offset]!

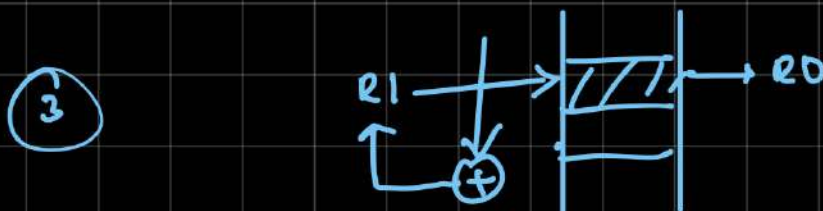
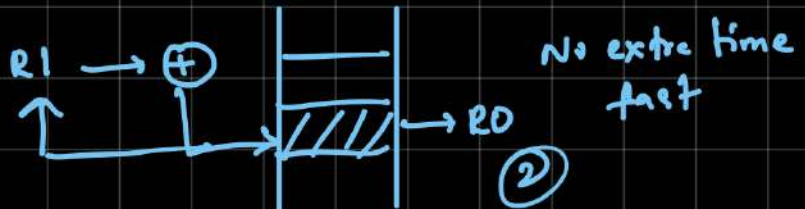
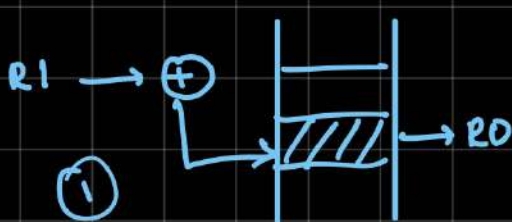
POST INDEXING

LDR Rd, [Rn], offset

pre-indexing: LDR R0, [R1, R2] @ R0 = mem[R1+R2]
R1 unchanged

auto indexing: LDR R0, [R1, R2]! @ R0 = mem[R1+R2]
R1 = R1+R2

post-indexing: LDR R0, [R1], R2 @ R0 = mem[R1]
R1 = R1+R2



- Program to find the sum of N NUMBERS
- Store the result in memory location.

Program:

.DATA	L1:	LDR R3, [R1, #4]	
A: .WORD 10,20,30,40,50,60,70,80,90,100		ADD R4,R4,R3	; Add next element in the array.
SUM: .WORD 0		ADD R1,R1,#4	
.TEXT		ADD R5, R5, #1	; increment the count register
LDR R1,=A		CMP R5, #11	; Check whether all numbers are added
LDR R2=SUM		BNE L1	; Else branch to L1 location
MOV R4,#0 ; INITIALISATION		STR R4,[R2]	; store the result in location SUM.
MOV R5,#1 ; COUNT register		SWI 0X011	; logical end of the program.

Has no impact on the preindexing mode....

- Program to find the sum of N NUMBERS
- Store the result in memory location.

Program:

.DATA	L1:	LDR R3, [R1, #4]!	
A: .WORD 10,20,30,40,50,60,70,80,90,100		ADD R4,R4,R3	; Add next element in the array.
SUM: .WORD 0		ADD R5, R5, #1	; increment the count register
.TEXT		CMP R5, #11	; Check whether all numbers are added
LDR R1,=A		BNE L1	; Else branch to L1 location
LDR R2=SUM		STR R4,[R2]	; store the result in location SUM.
MOV R4,#0 ; INITIALISATION		SWI 0X011	; logical end of the program.
MOV R5,#1 ; COUNT register			

- Program to find the sum of N NUMBERS and Store the result in memory location.

Program:

.DATA	L1:	LDR R3, [R1], #4
A: .WORD 10,20,30,40,50,60,70,80,90,100		ADD R4,R4,R3
SUM: .WORD 0		ADD R5,R5,#1
.TEXT		CMP R5, #11
LDR R1,=A		BNE L1
LDR R2=SUM		STR R4,[R2]
MOV R4,#0 ; INITIALISATION		SWI 0X011
MOV R5,#1 ; COUNT register		

Index methods.

Index method	Data	Base address register	Example
Preindex with writeback	$mem[base + offset]$	$base + offset$	LDR r0, [r1, #4]!
Preindex	$mem[base + offset]$	not updated	LDR r0, [r1, #4]
Postindex	$mem[base]$	$base + offset$	LDR r0, [r1], #4

Note: ! indicates that the instruction writes the calculated address back to the base address register.

Q. Assume $r0 = 0x20000000$, $r1 = 0x87654321$, Little endian. Find memory contents after the following program completes

STR r1, [r0, #4]!
 STR r1, [r0], #4
 STRH r1, [r0, #4]

H → half word.
 little — lower half
 big — upper half

PRE $r0 = 0x00000000$
 $r1 = 0x00090000$
 $mem32[0x00009000] = 0x01010101$
 $mem32[0x00009004] = 0x02020202$

LDR r0, [r1, #4]!

Preindexing with writeback:

POST(1) $r0 = 0x02020202$
 $r1 = 0x00009004$

LDR r0, [r1, #4]

Preindexing:

POST(2) $r0 = 0x02020202$
 $r1 = 0x00009000$

LDR r0, [r1], #4

Postindexing:

POST(3) $r0 = 0x01010101$
 $r1 = 0x00009004$

Address	Contents
0x2000,0013	
0x2000,0012	
0x2000,0011	
0x2000,0010	
0x2000,000F	
0x2000,000E	
0x2000,000D	0x43
0x2000,000C	0x21
0x2000,000B	
0x2000,000A	
0x2000,0009	
0x2000,0008	
0x2000,0007	0x87
0x2000,0006	0x65
0x2000,0005	0x43
0x2000,0004	0x21
0x2000,0003	
0x2000,0002	
0x2000,0001	
0x2000,0000	

SESSION
5A.

MULTIPLICATION INST.

MLA {cond}{S} Rd, Rm, Rs, Rn

MUL {cond}{S} Rd, Rm, Rs

MLA — multiply and accumulate $Rd = (Rm \times Rs) + Rn$
MUL — multiply $Rd = Rm \times Rs$

<inst> {cond}{S} RdLo, RdHi, Rm, Rs

SMLAL	signed multiply accumulate	$[RH:RL] = [RH:RL] + (Rm \times Rs)$
SMULL	signed multiply long	$[RH:RL] = Rm \times Rs$
UMLAL	unsigned multiply accumulate	$[RH:RL] = [RH:RL] + (Rm \times Rs)$
UMULL	unsigned multiply long	$[RH:RL] = Rm \times Rs$

- * Second op can't be immediate
- * res. reg should be diff from first op
- * Cycles depend on core type
- * If S bit is set, C flag is meaningless

Maximum Result of Multiplication

For 2 unsigned 64-bit integers,

$$(2^{64} - 1) \times (2^{64} - 1) = 2^{128} - 2^{65} + 1$$

will not exceed 128 bits

MLA - MUL
Questions

LSb $\rightarrow$ $8 - 2^0$

CONDITIONAL EXECUTION

Mnemonic	Condition	Mnemonic	Condition
CS	<i>Carry Set</i>	CC	<i>Carry Clear</i>
EQ	<i>Equal (Zero Set)</i>	NE	<i>Not Equal (Zero Clear)</i>
VS	<i>Overflow Set</i>	VC	<i>Overflow Clear</i>
GT	<i>Greater Than</i>	LT	<i>Less Than</i>
GE	<i>Greater Than or Equal</i>	LE	<i>Less Than or Equal</i>
PL	<i>Plus (Positive)</i>	MI	<i>Minus (Negative)</i>
HI	<i>Higher Than</i>	LO	<i>Lower Than (aka CC)</i>
HS	<i>Higher or Same (aka CS)</i>	LS	<i>Lower or Same</i>

Questions
(21-31)

Q. Perform instruction encoding for
 ADDS R1, R0, R2 LSL #2

31	28 27	26 25	24	21 20	19	16 15	12 11	7 6	5	4 3	0
1110	00	0	0100	1	0000	0001	00010	01	0	0010	

0xe0901122

SUBS R1, R2, R3 LSL #5

31	28 27	26 25	24	21 20	19	16 15	12 11	7 6	5	4 3	0
1110	00	0	0010	1	0010	0001	00101	00	0	0011	

0xe0521283

ADDS R1, R0, R2 LSR R4

31	28 27	26 25	24	21 20	19	16 15	12 11	8 7	6	5	4	3	0
1110	00	0	0100	1	0000	0001	0100	0	01	1	0010		

0xe0901432

BRANCH INSTRUCTION ENCODING



↓
branch

L → 1 branch link
 0 branch not link

$$\text{offset} = \frac{\text{Target Address} - (\text{PC} + 8)}{4}$$

$$\text{Target address (label)} = (\text{PC} + 8) + \underset{\substack{\downarrow \\ \text{offset} \times 4}}{\text{offset} \ll 2}$$

Q. 00001000 : foo

```
0000100c : bl foo
```

$$\text{offset} = \frac{0 \times 1000 - 0 \times 1014}{4} = \left(-\frac{14}{4}\right)_{16} = \left(-\frac{20}{4}\right)_{10} = (-5)_{10}$$



Q. Calculate instruction encoding for branch instruction
 (i) 100c B forward (2) 102c B backward
 101c forward: 1020 backward:

$$(1) \text{ offset} = \frac{0x101c - (0x100c + 8)}{4} = \frac{8}{4} = 2$$

31	28	27	26	25	24	23																			0									
1	1	1	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

0xea000002

$$(2) \text{ offset} = \frac{0x1020 - (0x102c + 8)}{4} = \frac{(-14)}{4} = \frac{(-20)}{4} = (-5)_{10}$$

31	28	27	26	25	24	23																			0								
1	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1

0xeaffffffb

MULTIPLICATION INST. ENCODING

Q. MUL R0, R1, R2 → mul rd, rn, rs

31	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

0xe0000291

Q. MLA R0, R1, R2, R3 → mla rd, rn, rs, rn

31	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
1	1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

0xe0203291

Second type. $\rightarrow$ Signed & unigned for mul

Q. SMULL R0, R1, R2, R3 $\rightarrow$ Rdlo, Rdhi, Rm, Rs

1110	00001	1	0	0	0001	0000	0011	1001	0010
------	-------	---	---	---	------	------	------	------	------

0xE0C10392

Q. UMLAL R0, R1, R2, R3 $\rightarrow$ rdlo, rdhi, rm, rs

1110	00001	0	1	0	0001	0000	0011	1001	0010
------	-------	---	---	---	------	------	------	------	------

0xE0A10392